MP3: Page Table Management

Nithin Gowtham Saravanan
UIN: 337001954
CSCE611: Operating System

Assigned Tasks

Main: Completed.
Bonus Option 1: Demonstration of the implemented design
Bonus Option 2: Not applicable.
Bonus Option 3: Not applicable.
Bonus Option 4: Not applicable.

System Design

The objective of this machine problem is to implement a demand-paging virtual memory system for the kernel on x86 architecture, supporting a single address space with future support for multiple processes. This entire project is built on top of Machine Problem 2 which already has contiguous frame pool implementation.

Memory breakdown:

1. Between 0 and 4MB: Kernel Pool of which we use frames located between 3MB – 4MB only
2. Between 4MB and 32MB: Process pool

Page Table Design:

1. First frame in kernel pool (after 2MB) – Page Directory (L1 in Page table management)
2. Second frame in kernel pool (after 2MB) – First table of page table pages that hold the memory reference for kernel pool. Since kernel pool has 1024 frames in total, the available 1024 pages in the first page will be completely dedicated to kernel. This means that entry 0 of the PDE points to Page table 0 that holds kernel frames.
3. Third frame in kernel pool (after 2MB) will have the second page table page and starts to store the frames referenced by process pool for up to 1024 frames in that page table page and so on.

Method of Implementation:

1. Initializing frame pools - please refer to the design.pdf of MP2 for the implementation of cont_frame_pool.H and cont_frame_pool.C
2. init-paging: To set the parameters for the paging subsystem.
3. PageTable object creation: As part of this object creation, a constructor is executed that sets up the entries in the page directory and the page table. The page table entries for the shared portion of the memory (i.e. the first 4MB) must be marked valid ("present" in x86 parlance).
4. After the page table is created, the kernel loads it into the processor context through the load() function.
5. Till step 4, paging is not enabled but just setup. Now enable paging so that the process pool can use the page table management
6. Page fault handler – Whenever a page fault occurs (Exception 14),
    a. It finds a free frame from a common free-frame pool
    b. It allocates this new frame to the process
    c. The page entry is updated accordingly
    d. The page fault handler returns
    e. The CPU then retries the instruction, and this time finds the physical-to-virtual mapping ready to be used in the paging data structures.

Code Description

This implementation modifies the following files:

- cont_frame_pool.H
- cont_frame_pool.C
- page_table.C
- kernel.C

cont_frame_pool.H / cont_frame_pool.C

The files developed for Machine Problem 2 are used here for page table management implementation. A detailed description of this implementation is available at the MP2_Sources\design.pdf from https://github.com/tamu-edu-students/CSCE410-611-Fall2025-Nithin-Gowtham-Saravanan.git

kernel.C

This file has been updated with necessary commands to be displayed at the output screen for better understanding of the stages executed by kernel. No functional modifications are made.

page_table.C: Initialization and #defines

- Define the actual storage for static members declared in the header to 0 or nullptr (whichever is applicable)
- Define the bits in #define for page present, page r/w and page to be used in kernel/user pool

page_table.C: init_paging

- Executed once at the beginning of kernel.C
- This saves pointers for kernel and process frame pools
- Saves the shared size memory area (in our case its 4MB)

page_table.C: PageTable() – Constructor

- Allocate one frame for page directory from kernel pool (considered as first level in Page management). This can handle 1024 entries, meaning it can point to 1024 page table pages in which each has 1024 entries in it.
- Compute the number of shared pages and PDE needed
- Allocate one page table from the kernel pool.
- Fill entries so that **virtual = physical** for the shared region (directly mapped) and mark the entries as present and writable.
- Add Page Table's frame address of the shared region to the PDE (here 0^{th} entry for first 4MB)
- Set this page table as the current page table (for pointing)

page_table.C: load()

- Writes the physical address of PDE to CR3
- Sets the global current page table
- Ready for paging to be enabled

page_table.C: enable_paging()

- Sets bit 31 of CR0 to enable paging and software flag for paging_enabled is set to TRUE

page_table.C: handle_fault()

- Kernel is designed such that whenever exception 14 occurs, the fault address is loaded to register CR2
- Extract the PDE and PTE bits from the address (i.e.) bits 31-22 and bits 21-12 respectively
- Check if an entry for PDE exists. If no page table exists, create one entry and update the page directory entry to point to it
- Map the faulty page. If the PTE for that page is not present, allocate that frame from process memory pool and set the PTE to point to the frame with flags Page_Present and PageRW
- Invoke invlpg() to flush any cached TLB entries for this address. *(Note: This flushing is a technique inferred from google for better reliability)*
- If the entry already exists and the page fault is still triggered, it might be probably due to an access violation. Eg: 15MB-16MB that should be inaccessible

Bonus Option 1:

Demonstration of the implemented design

- Each stage of execution in kernel.C is updated with valid commands, and the entire process flow is captured in the log report **kernel_log.txt**
- This log report is generated using the command **make run > kernel_log.txt 2>&1**

Testing

Testing involved

- Initialization of contiguous frame pool for kernel and process pools
- Page Table Setup
- Loading the Page Table
- Enabling Paging
- Access shared memory to ensure directly mapped pages are working
- Access unshared memory to trigger page faults and allocate frames in PTE

Note: The screenshots are attached from the kernel_log.txt

Test Procedure

1. **Install the exception handler and interrupt handler**

```
Installing handler in IDT position 0
Installing handler in IDT position 1
Installing handler in IDT position 2
Installing handler in IDT position 3
Installing handler in IDT position 4
Installing handler in IDT position 5
Installing handler in IDT position 6
Installing handler in IDT position 7
Installing handler in IDT position 8
Installing handler in IDT position 9
Installing handler in IDT position 10
Installing handler in IDT position 11
Installing handler in IDT position 12
Installing handler in IDT position 13
Installing handler in IDT position 14
Installing handler in IDT position 15
Installing handler in IDT position 16
Installing handler in IDT position 17
Installing handler in IDT position 18
Installing handler in IDT position 19
Installing handler in IDT position 20
Installing handler in IDT position 21
Installing handler in IDT position 22
Installing handler in IDT position 23
Installing handler in IDT position 24
Installing handler in IDT position 25
Installing handler in IDT position 26
Installing handler in IDT position 27
Installing handler in IDT position 28
Installing handler in IDT position 29
Installing handler in IDT position 30
Installing handler in IDT position 31
Installing handler in IDT position 32
Installing handler in IDT position 33
Installing handler in IDT position 34
Installing handler in IDT position 35
Installing handler in IDT position 36
Installing handler in IDT position 37
Installing handler in IDT position 38
Installing handler in IDT position 39
Installing handler in IDT position 40
Installing handler in IDT position 41
Installing handler in IDT position 42
Installing handler in IDT position 43
Installing handler in IDT position 44
Installing handler in IDT position 45
Installing handler in IDT position 46
Installing handler in IDT position 47
Installed exception handler at ISR <0>
Installed interrupt handler at IRQ <0>
```

2. **Initialization of kernel memory pool**
    a. This initializes the contiguous frame pool between 2MB and 4MB

```
Setting up kernel memory pool
Contiguous Frame Pool initialized
```

3. **Initialization of process pool**
    a. This initializes the process pool between 4MB and 32MB
    b. Checks the number of frames to store the process pool info
    c. Searches for available frames in process pool and initializes the contiguous frame pool

```
Setting up process memory pool
ContFramePool::needed_info_frames: frames needed = 1

Frame to store process pool info
Executing get_frames
Success: found contiguous block from local frames 1 to 1
Contiguous Frame Pool initialized
```

4. **Marking inaccessible memory (15MB-16MB)**
    a. To ensure that frames in this memory range is not used, we set the frame status as “used”
    b. The frame numbers correspond to the frames in the specified memory range

```
Inaccessible memory 15MB-16MB
Marking inaccessible memory range: 3840 - 4095 (frame numbers)
```

5. **Intializing Paging System**
    a. To map the pool registration data and shared size information

```
Initialized Paging System
Kernel pool and process pool registered
Shared region size is set to 4 MB
```

6. **Creating an object for PageTable and loading the constructor**
    a. Checks for frame in kernel pool to store page directory
    b. Used 1 frame in kernel pool as PDE with 1024 entries(L1) that can point up to 1024 page table pages (1024). This will ideally be the 2nd frame in kernel pool from 2MB
    c. Once allocated, it creates the first page table in the 3rd frame pointing to the corresponding PDE. Thus first page table has data of directly mapped address locations

```
Page Table Constructor
Executing get_frames
Success: found contiguous block from local frames 2 to 2
Page Directory frame is allocated in kernel pool
Shared region requires <1024> pages across <1> page directory entries
Executing get_frames
Success: found contiguous block from local frames 3 to 3
Built Page Table <0> for shared region
Constructed Page Table object
```

7. **Loading Page Table**
    a. To load PDE into CR3 and activate the page table

```
Loading Page Table
Page Directory loaded into CR3
This Page Table is now active
Loaded page table
```

8. **Enabling Paging**
    a. To enable paging in CR0 register. Till the previous step, the concept of paging is developed but not put in use yet. From now on, process pool frames will use page table entries

```
Enabling Paging in CR0
PG bit set, paging is now enabled!

WE TURNED ON PAGING!

If we see this message, the page tables have been
set up mostly correctly.
Hello World!
```

9. Triggering Page Faults

a. A frame from process pool is accessed. A page fault is triggered
b. Since the first page table page is filled with pages of kernel pool, the faulted page should be allocated into a different page table. Now, a new page table is created and it's linked with the PDE. It analyzes the page is not present and assigns it into the first frame of the new page table
c. Likewise, the page faults are handled. In our case, we try to access 256 pages from process pool and one new page table in addition to kernel pool page table is enough. Since a page table page has 1024 entries and we are accessing only 256 pages from process pool now.
d. The first page fault when triggered, performs page table creation and page allocation in the created page table

```
EXCEPTION DISPATCHER: exc_no = <14>
Page Fault triggered
Missing Page Table. Allocating new one...
Executing get_frames
Success: found contiguous block from local frames 4 to 4
New Page Table allocated and linked
Page not present. Allocating data frame...
Executing get_frames
Success: found contiguous block from local frames 0 to 0
Mapped Faulted Page #: <1024> from physical memory (process pool) to entry #: <0> of page table #: <1>
Page Fault successfully handled
```

This clearly states that 1024th frame (i.e. first frame in process pool after 0-1023 frames in kernel pool) is stored in entry 0 of page table #1.
Note: page table #0 has pages of kernel pool. Frames 4 to 4 is the next frame number in kernel pool for the new page table. Frame 0 of 0 is the first frame in process pool.

e. For the next page fault

```
EXCEPTION DISPATCHER: exc_no = <14>
Page Fault triggered
Page not present. Allocating data frame...
Executing get_frames
Success: found contiguous block from local frames 2 to 2
Mapped Faulted Page #: <1026> from physical memory (process pool) to entry #: <2> of page table #: <1>
Page Fault successfully handled
```

f. In a similar way, this happens till 256th entry

```
EXCEPTION DISPATCHER: exc_no = <14>
Page Fault triggered
Page not present. Allocating data frame...
Executing get_frames
Success: found contiguous block from local frames 255 to 255
Mapped Faulted Page #: <1279> from physical memory (process pool) to entry #: <255> of page table #: <1>
Page Fault successfully handled
```

10. Check Memory references

a. Once all the page faults are handled, validate the value in the memory location if it is different from the value we entered earlier
b. The qemu is terminated forcefully after execution

```
DONE WRITING TO MEMORY. Now testing...
TEST PASSED
YOU CAN SAFELY TURN OFF THE MACHINE NOW.
One second has passed
One second has passed
One second has passed
One second has passed
One second has passed
qemu: terminating on signal 2
```

Note: All tests were performed using the provided console print utilities and assertion checks.